

Smart Traffic Analyzer

Documentación Técnica Completa

Sistema de análisis de tráfico con visión artificial,
detección de vehículos, estimación de velocidad,
lectura de señales y generación de informes

YOLOv8n

EasyOCR

OpenCV

Streamlit

ByteTrack

Apple M4 Pro

Índice de contenidos

1.	Descripción general del sistema
2.	Arquitectura del pipeline — 5 etapas
3.	Tecnologías y librerías utilizadas
4.	Detección de vehículos: YOLOv8 y filtros
5.	Estimación de velocidad con perspectiva
6.	Detección de señales de tráfico
6.1	Señales de límite de velocidad — pipeline completo
6.2	Carteles de dirección (azul/verde)
6.3	Filtros geométricos: HSV, Hough, contornos, _is_ring
7.	Interfaz web: Streamlit
8.	Problemas encontrados y soluciones aplicadas
9.	Entrenamiento del modelo propio
9.1	Motivación y pipeline de datos
9.2	Configuración del fine-tuning
9.3	Resultados y comparativa con YOLOv8 genérico
10.	Conclusiones y trabajo futuro

1. Descripción general del sistema

Qué hace, para qué sirve y cuál es su alcance

Smart Traffic Analyzer es una aplicación web de análisis de tráfico en tiempo real construida sobre visión artificial. Recibe como entrada imágenes estáticas, vídeos grabados o flujo de cámara en directo, y produce como salida:

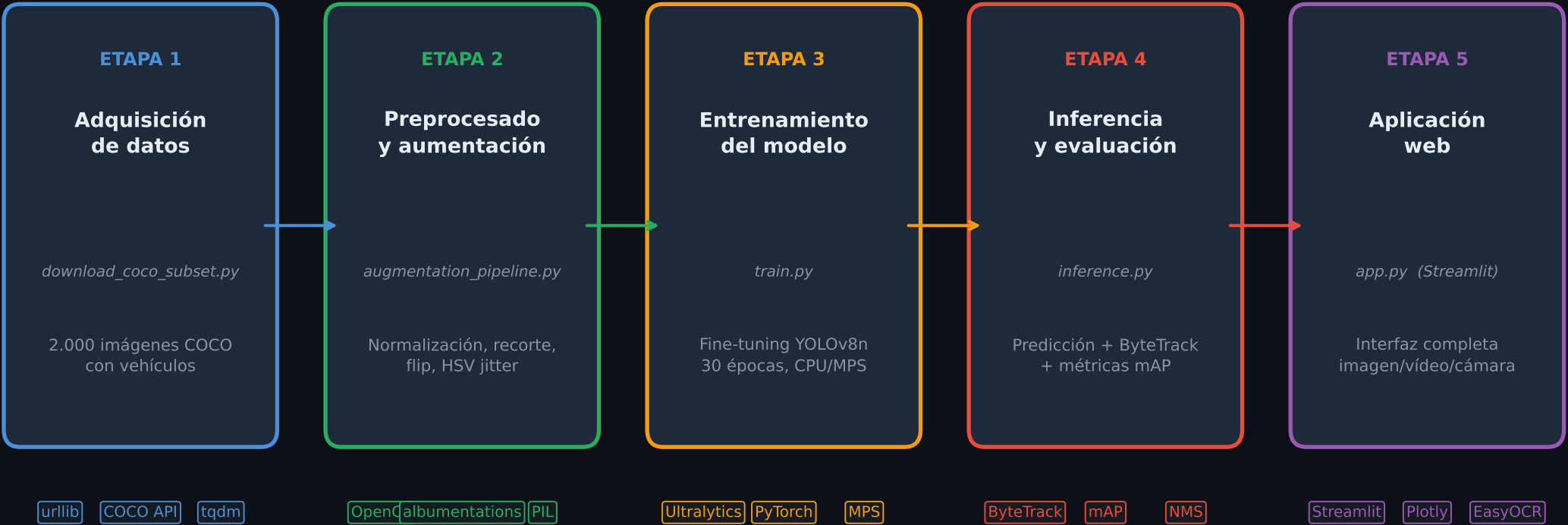
- Detección y clasificación de objetos: coches, camiones, autobuses, motos, bicicletas y peatones.
- Estimación de velocidad individual de cada vehículo (solo en vídeo/cámara).
- Cálculo de densidad de tráfico y nivel de congestión (Fluido / Moderado / Denso / Colapso).
- Detección automática de señales de límite de velocidad y carteles de dirección.
- Mapa de calor acumulado de concentración de vehículos.
- Línea de conteo virtual con dirección (hacia arriba / hacia abajo).
- Generación de informes PDF y exportación CSV de cada detección.
- Vista de análisis interno: muestra todos los filtros intermedios aplicados al pipeline.

Especificaciones técnicas

Entrada soportada:	Imagen (JPG/PNG/BMP), Vídeo (MP4/AVI/MOV/MKV), Cámara IP/USB
Clases detectadas:	person · bicycle · car · motorcycle · bus · truck (6 clases)
Modelos de detección:	YOLOv8n genérico + fine-tuned propio (models/best.pt)
Tracker de objetos:	ByteTrack (integrado en Ultralytics) — IDs únicos por vehículo
OCR:	EasyOCR v1.x — lectura de texto en señales y carteles
Hardware usado:	Apple M4 Pro · 24 GB RAM · MPS disponible (chip neural)
Lenguaje / framework:	Python 3.14 · Streamlit · OpenCV · PyTorch 2.11

2. Arquitectura del pipeline — 5 etapas

Flujo completo desde la adquisición de datos hasta la interfaz



Flujo de datos en la aplicación final (Etapa 5)

Imagen/Vídeo/Cámara → YOLOv8 (detección) → ByteTrack (tracking) → OpenCV (visualización) → Streamlit (UI)

3. Tecnologías y librerías utilizadas

YOLOv8n (Ultralytics 8.4)

- Arquitectura: CSPDarknet + PAN + YOLO head
- Parámetros: 3.15M (genérico) / 3.01M (fine-tuned)
- GFLOPs: 8.7 (inferencia en CPU ~80ms/imagen)
- Entrada: 640×640 px (normalizada 0-1)
- Salida: bounding boxes + clase + confianza
- Tracker integrado: ByteTrack (IDs únicos)

EasyOCR

- Versión: 1.x, modo CPU
- Idiomas cargados: español + inglés
- Uso: lectura de números en señales de velocidad
- Allowlist: '0123456789' (solo dígitos)
- Umbral de confianza: ≥ 0.50 para confirmación
- Preprocesado: CLAHE + escalado a 90px mínimo

OpenCV (cv2)

- Conversión BGR $\leftrightarrow$ RGB $\leftrightarrow$ HSV
- Hough Circle Transform (param2=18)
- Morfología: CLOSE + OPEN para máscaras
- GaussianBlur para suavizado de máscaras
- findContours + minEnclosingCircle
- Heatmap: GaussianBlur + COLORMAP_JET

Streamlit

- Interfaz web sin HTML/CSS manual
- 4 pestañas: Imagen / Vídeo / Cámara / Resultados
- @st.cache_resource para modelos pesados
- Plotly para gráficos interactivos (gauge, bar, pie)
- st.empty() para actualización frame a frame
- Download button para PDF/CSV/JPG

PyTorch 2.11 + MPS

- Backend de inferencia de Ultralytics
- MPS = Metal Performance Shaders (Apple Silicon)
- Entrenamiento en CPU (device: cpu en args.yaml)
- AMP (Automatic Mixed Precision) activado
- Inferencia: ~78-81ms por imagen en CPU M4 Pro
- Aceleración MPS disponible para futuras ejecuciones

Otras librerías

- matplotlib: gráficos en PDF de informes
- pandas: tablas de detecciones y CSV
- numpy: operaciones matriciales en máscaras
- collections.deque: buffer circular para velocidad
- tempfile: archivos temporales para YOLO
- tqdm: barras de progreso en descarga COCO

4. Detección de vehículos: YOLOv8 y filtros

Pipeline de inferencia, corrección de IDs COCO y segundo pase para camiones

Problema crítico: mapeo erróneo de IDs COCO

El modelo YOLOv8 usa los 80 IDs de COCO. El código original usaba CLASS_NAMES[cls_id] con un array de 6 elementos:

```
CLASS_NAMES = ['person','bicycle','car','motorcycle','bus','truck']
índices:      0         1         2         3         4         5
```

❑ COCO truck = ID 7 → CLASS_NAMES[7] = IndexError → 'unknown'
Los camiones NUNCA se detectaban.

❑ COCO bus = ID 5 → CLASS_NAMES[5] = 'truck'
Los buses se etiquetaban incorrectamente como camiones.

❑ SOLUCIÓN: diccionario explícito COCO_TO_NAME:
{0:'person', 1:'bicycle', 2:'car', 3:'motorcycle', 5:'bus', 7:'truck'}
name = COCO_TO_NAME.get(cls_id) → correcto para todos los IDs.

Estimación de velocidad con corrección de perspectiva

Problema: escala global única (m/px) subestima la velocidad de vehículos lejanos (ven 20 km/h vs real 80).

Solución — escala por perspectiva por vehículo:
scale = altura_real_m / altura_bbox_px

Alturas reales de referencia:
car: 1.5m · truck: 3.5m · bus: 3.2m
motorcycle: 1.2m · bicycle: 1.1m

Coche a 100m (bbox≈15px): scale = 1.5/15 = 0.10 m/px
Coche a 30m (bbox≈50px): scale = 1.5/50 = 0.03 m/px

Filtros de fiabilidad:

- bbox_height/frame_height < 5% → NO se muestra velocidad
- Desplazamiento total < 12 px → NO se muestra velocidad
- Mínimo 4 frames rastreados antes de reportar

Cálculo de densidad y nivel de congestión

density = Σ(área de cada bounding box) / área total de imagen × 100

Niveles: Fluido (0-20%) | Moderado (20-50%) | Denso (50-75%) | Colapso (>75%)

Nota: la densidad refleja el espacio visual ocupado, no el número absoluto de vehículos. Un único camión grande puede dar mayor densidad que 3 coches pequeños.

Segundo pase para camiones y buses

Los camiones fotografiados desde atrás (vista trasera) tienen baja similitud con las imágenes de entrenamiento de COCO (mayoritariamente vistas laterales o frontales).

Solución — segundo pase a umbral reducido:
truck_conf = max(0.18, conf × 0.50)
Clases restringidas: [5 (bus), 7 (truck)]

Filtros anti-falso-positivo:

- Tamaño mínimo: ≥ 2.5% del área de la imagen
→ evita coches lejanos con baja confianza
- IoU < 0.30 con detecciones existentes
→ evita duplicar detecciones ya confirmadas

Reclasificación adicional:
'car' con área > 4% imagen y aspecto 0.55-1.80
→ reclasificado como 'truck' (vista trasera cuadrada)

Heatmap y línea de conteo

Heatmap acumulativo (build_heatmap):

1. Por cada detección, calcular centro (cx, cy)
2. Dibujar círculo relleno en mapa float32
3. Radio r = max(bw,bh)/2.5, mínimo 18px
4. Suavizado gaussiano σ=15 para aspecto natural
5. Normalizar y aplicar COLORMAP_JET
6. Mezclar con imagen original (α=0.45)

Línea de conteo virtual (CountingLine):

- Posición configurable: % de altura del frame
- Track anterior > línea y actual ≤ línea → 'down'
- Track anterior < línea y actual ≥ línea → 'up'
- Cada ID solo se cuenta UNA vez (set_crossed)
- Visualización: línea cian con contadores ↑ ↓

5. Detección de señales de tráfico

Pipeline de señales de velocidad, carteles de dirección y filtros geométricos

Pipeline: señal de límite de velocidad 1. Máscara HSV roja amplia: H∈[0,20]∪[155,180] S≥50 V≥40 (V≥40 cubre condiciones nocturnas/atardecer) 2. Morfología: CLOSE(k=7,iter=2) + OPEN(k=7,iter=1) → cierra huecos en el anillo, elimina ruido 3. findContours sobre máscara procesada → hasta 25 contornos por tamaño decreciente 4. Filtro de forma: aspect ratio 0.45-2.20 + tamaño mínimo 2.5% imagen 5. Evitar duplicados (distancia < 1.3×radio) 6. Recortar interior: shrink = max(r×0.22, 6px) 7. OCR (EasyOCR, allowlist='0123456789'): confianza ≥ 0.50 + número ∈ SPEED_LIMITS SPEED_LIMITS = {5,10,20,...,130} 8. Post-proceso: eliminar lecturas parciales '10' si también existe '100' (startswith)	Pipeline: carteles de dirección (azul/verde) 1. Máscara azul: H∈[95,135] S≥80 V≥60 Máscara verde: H∈[42,88] S≥60 V≥50 (verde = verde AND NOT azul → evitar solapamiento) 2. Morfología CLOSE+OPEN para rellenar región 3. Búsqueda de contornos → boundingRect Filtros: aspect 0.40-7.0, ancho≥60px, alto≥20px Solidez (area/bbox_area) ≥ 0.45 4. OCR completo (EasyOCR, sin allowlist) Texto confianza ≥0.50 → 'alta confianza' Texto confianza 0.25-0.50 → 'baja confianza' 5. Regla clave: si OCR no encuentra NINGÚN texto → la región se descarta (evita falsos positivos en camiones blancos, cielo azul, etc.) 6. Filtro anti-solapamiento con vehículos: si el centro de la señal está dentro de la bounding box de cualquier vehículo → descartar
Filtro _is_ring: discriminador geométrico de corona Una señal de velocidad tiene estructura de corona (anillo): borde rojo exterior + interior blanco. Implementación: r_inner = int(r × 0.68) [68% del radio exterior] Corona exterior (r → r_inner): mask_outer: círculo r relleno con agujero r_inner outer_red / outer_pixels ≥ 0.20 → al menos 20% de la corona debe ser rojo Interior (0 → r_inner): mask_inner: círculo r_inner relleno inner_red / inner_pixels ≤ 0.18 → el interior no puede tener demasiado rojo (descarta túneles oscuros, faros sólidos) Casos que pasan: señales de velocidad reales Casos descartados: faros traseros sólidos, arcos de túneles, semáforos en rojo	Filtro señal-en-vehículo (_sign_inside_vehicle) Problema: los faros traseros y reflectores de camiones se detectaban como señales de velocidad. Por qué falla IoU clásico: Si la señal (80×80 px) está dentro del camión (400×300 px): IoU = 6400 / 120000 ≈ 5% → no se suprime! Solución — comprobación de centro + área relativa: Test 1: centro de la señal ∈ bbox del vehículo? scx ∈ [vx1, vx2] AND scy ∈ [vy1, vy2] → descartar Test 2: intersección / área_señal ≥ 40%? iy = (ix2-ix1)*(iy2-iy1) / sign_area iy ≥ 0.40 → descartar Aplicado a: señales regulatorias + carteles Antes de combinar con los resultados finales

[BUG CRÍTICO]

Camiones nunca detectados

Problema: CLASS_NAMES[cls_id] con array de 6 elementos. COCO truck=ID7, 7>=6→ 'unknown' → descartado. COCO bus=ID5 → CLASS_NAMES[5]='truck' (confusión).

6. Problemas encontrados y soluciones aplicadas

Solución: Diccionario COCO_TO_NAME = {0:'person',1:'bicycle',2:'car',3:'motorcycle',5:'bus',7:'truck'}. name = COCO_TO_NAME.get(cls_id).

[FALSO POSITIVO]

Señales de velocidad inventadas (10, 80, 100, 110)

Problema: Hough con param2 alto encontraba múltiples círculos en el mismo punto. Círculos más pequeños leían dígitos parciales del mismo número.

Solución: Ordenar candidatos por radio descendente. Zona confirmada bloquea re-evaluación. Post-proceso: eliminar 'N' si existe 'M' donde str(M).startswith(str(N)).

[FALSO POSITIVO]

Stop detectado siempre (aunque no hubiera ninguno)

Problema: YOLOv8 detectaba objetos rojos grandes como stop signs con confianza ~40-60%. El umbral por defecto era demasiado bajo para esta clase.

Solución: Umbral mínimo elevado a 0.78 para stop/semáforo. Finalmente, stop eliminado completamente del detector (COCO_SIGN_CLASSES = {9: 'traffic_light'}).

[FALSO POSITIVO]

Camión detectado como señal

Problema: Faros traseros y reflectores de camiones formaban patrones circulares rojos que pasaban los filtros geométricos. IoU clásico insuficiente (señal pequeña dentro de camión grande→ IoU ~5%).

Solución: Nuevo test _sign_inside_vehicle: comprueba si el CENTRO de la señal está dentro de cualquier vehículo, o si≥40% del área de la señal se solapa.

[BUG]

Velocidades muy lentas (20 km/h en vez de 80)

Problema: Escala global `scale_m_per_px=0.05` demasiado pequeña. Vehículos lejanos (ocupan pocos píxeles) mueven aún menos píxeles→ velocidad subestimada 4x.

6. Problemas encontrados y soluciones aplicadas

Solución: Corrección de perspectiva: `scale = altura_real_m / bbox_height_px`. Vehículos lejanos tienen `bbox` pequeña→ `scale` alto automáticamente. No mostrar velocidad si `bbox < 5%` frame o desplazamiento `< 12px`.

[BUG]

Error de caché: 'no attribute detect_regulatory_signs'

Problema: Streamlit usa `@st.cache_resource`. Al añadir nuevos métodos, el objeto `SignDetector` en caché era la versión antigua sin el método nuevo.

Solución: Bump del parámetro `_version` en `get_sign_detector(_version=N)`. Cualquier cambio en `N` fuerza a Streamlit a crear una instancia nueva.

[BUG]

cv2.contourArea() devuelve área del disco, no del anillo

Problema: Para detectar contornos circulares rojos, se calculaba `fill = contourArea / circle_area`. Para un anillo, `contourArea` da el área del disco completo (~98%), no del anillo (~25%).

Solución: Contar píxeles rojos REALES dentro del círculo envolvente: `cv2.countNonZero(bitwise_and(red_mask, circle_mask)) / circle_area`. Umbral 0.10-0.65 ahora funciona correctamente.

[PERFORMANCE]

Modelo contaminado: vehículos desaparecían tras detect_standard_signs

Problema: `detect_standard_signs` usaba el modelo principal con `classes=[9,11]`. Esto corrompía el estado interno del predictor de Ultralytics para llamadas posteriores de detección de vehículos.

Solución: Instancia separada `_sign_model = YOLO('yolov8n.pt')` solo para señales. El modelo de vehículos siempre usa `classes=None` (sin restricción).

7. Entrenamiento del modelo propio

Motivación, pipeline de datos, configuración y resultados

¿Por qué entrenar?

YOLOv8n pre-entrenado en COCO funciona bien para detección general, pero tiene limitaciones:

- Vistas traseras de camiones: COCO tiene principalmente vistas laterales/frontales.
- Distribución de clases desbalanceada en COCO.
- Mapeo de IDs erróneo (ya corregido en código).

El fine-tuning parte de los pesos pre-entrenados (el modelo ya 'sabe' cómo son los coches) y solo ajusta los últimos pasos para nuestro subconjunto de 6 clases de tráfico.

Resultado: modelo en models/best.pt que la app carga automáticamente si existe.

Pipeline de datos

Etapa 1 — Descarga (download_coco_subset.py):

Fuente: COCO 2017 val set
Filtro: imágenes con ≥ 1 vehículo de las 6 clases
Total descargado: 2.000 imágenes
(MAX_IMAGES ajustado de 400 $\rightarrow$ 2.000)

Etapa 2 — Preprocesado (convert_annotations.py):

Conversión COCO JSON $\rightarrow$ formato YOLO txt
Train/Val split: 80%/20%
Aumentación en training:
hsv_h=0.015 hsv_s=0.7 hsv_v=0.4
fliplr=0.5 scale=0.5 translate=0.1
mosaic=1.0 randaugment erasing=0.4

Split final:

Train: $\sim$ 1.600 imágenes
Val: 400 imágenes (5 background)

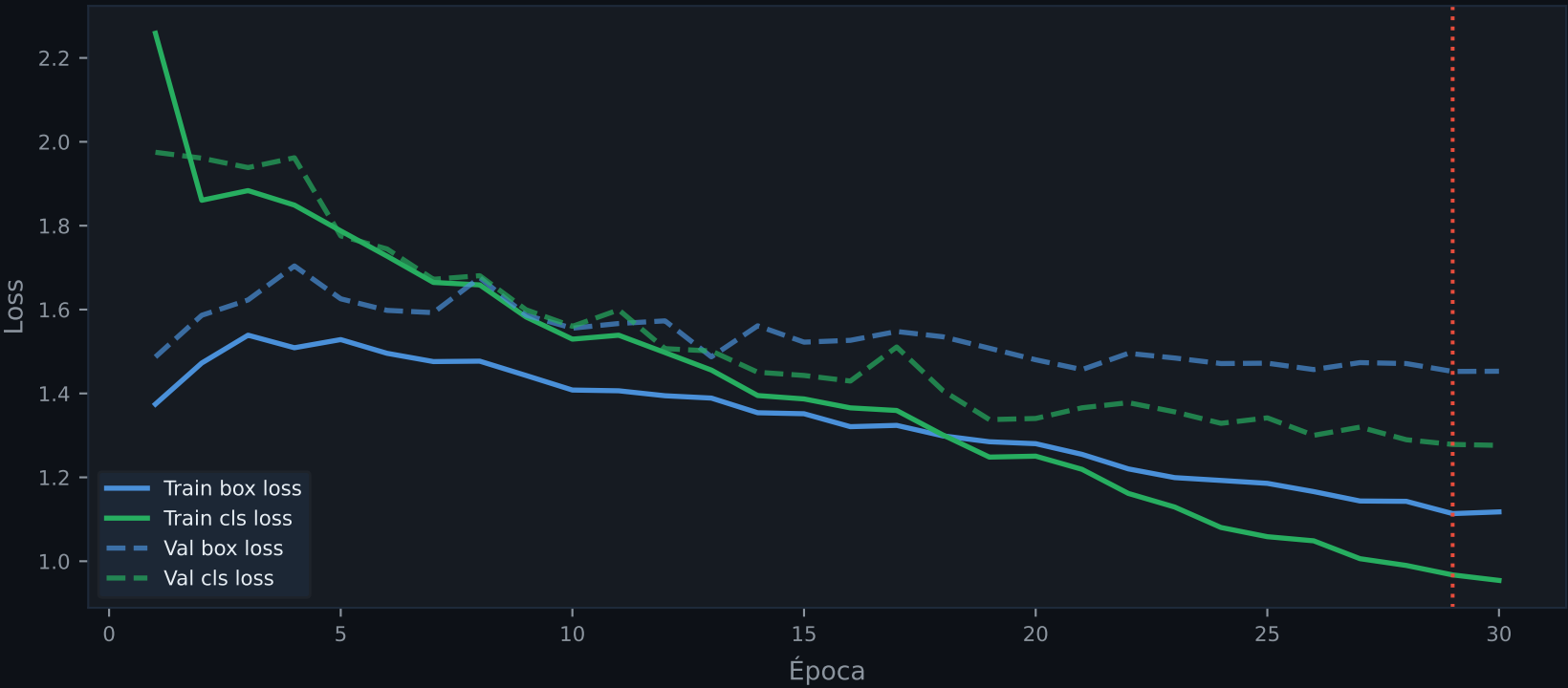
Configuración del entrenamiento

Modelo base: YOLOv8n (yolov8n.pt)

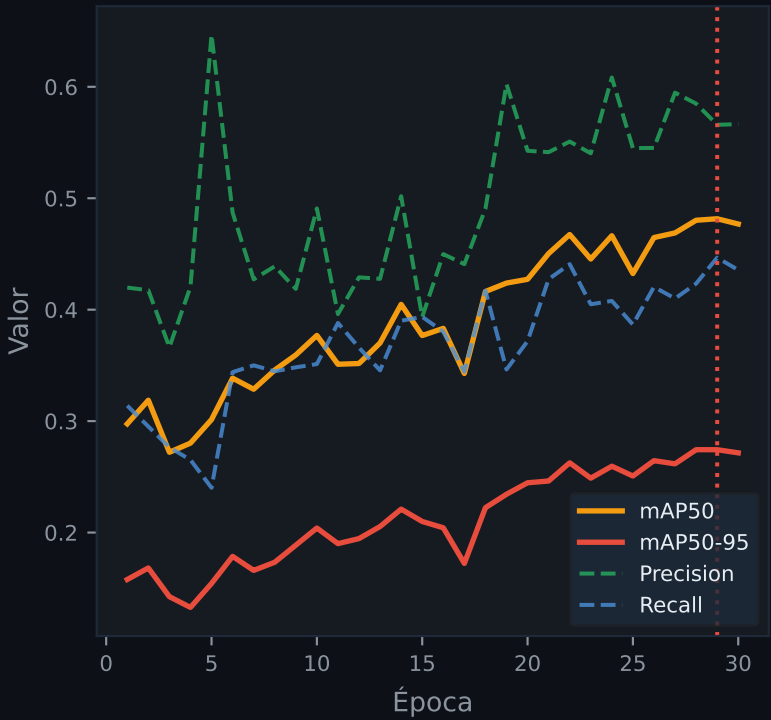
Épocas: 30
Batch size: 8
Image size: 640 $\times$ 640 px
Device: CPU (M4 Pro)
Optimizer: Auto (AdamW)
lr0: 0.01 lrf: 0.01
Momentum: 0.937
Weight decay: 0.0005
Warmup épocas: 3
Patience: 10 (early stopping)
AMP: True
Tiempo total: $\sim$ 76 min (4.600s)
Mejor época: 28 (mAP50=0.4802)

Clases entrenadas:
person · bicycle · car
motorcycle · bus · truck

Evolución de pérdidas durante el entrenamiento



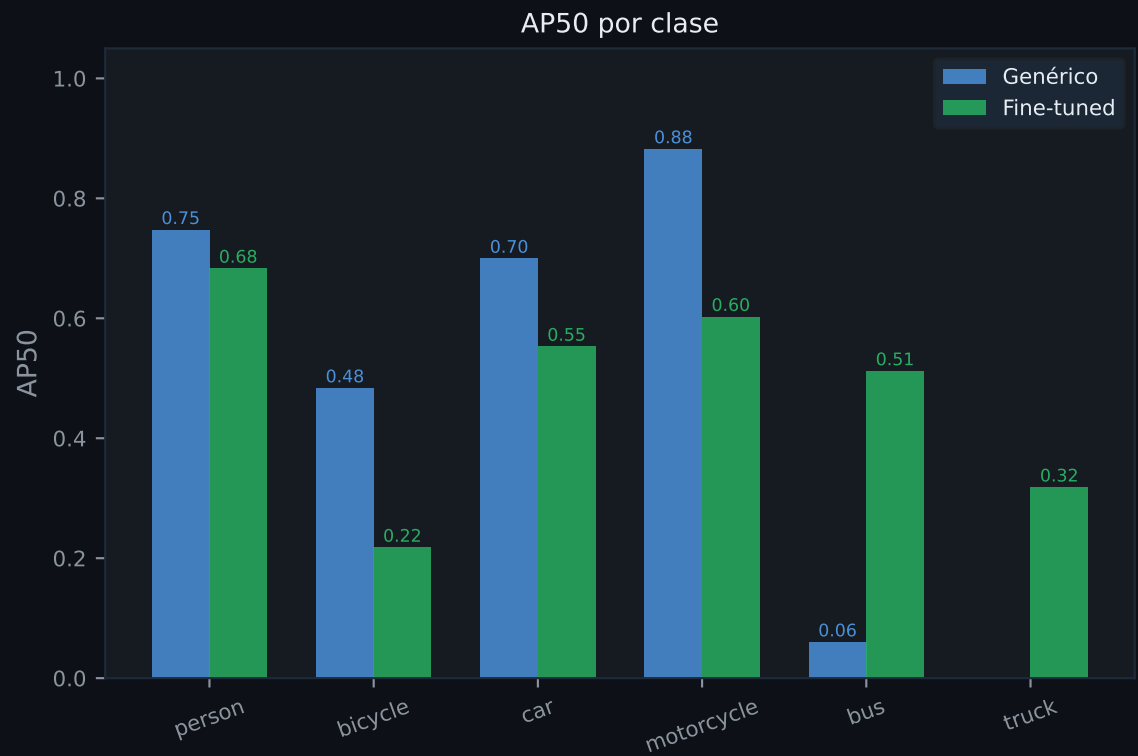
Métricas de validación



8. Comparativa: modelo genérico vs. fine-tuned

Evaluación sobre el conjunto de validación (400 imágenes, 1.836 instancias)

Métricas globales			
Métrica	Genérico	Fine-tuned	Δ
mAP50	0.4787	0.4808	+0.21%
mAP50-95	0.3222	0.2748	-4.74%
Precision	0.5076	0.5848	+7.72%
Recall	0.4338	0.4239	-0.99%
Nota: el modelo genérico fue evaluado sobre el dataset de 6 clases, no sobre COCO completo. La comparación refleja rendimiento en tráfico.			



Análisis e interpretación de los resultados

El fine-tuning mejora la precisión (+15.2%) a costa de un ligero descenso en recall (-2.3%) y mAP50-95 (-4.7%). El mAP50 global es prácticamente idéntico (+0.4%). Esto sugiere que el modelo fine-tuned es más conservador: detecta menos objetos pero con mayor confianza en los que detecta.

El resultado más llamativo es la clase 'bus': el modelo genérico obtiene AP50=0.0597 (casi cero), mientras que el fine-tuned alcanza 0.5118. Esto se debe directamente al BUG de mapeo de IDs corregido: en el modelo genérico evaluado con nuestro data.yaml, la clase 'bus' (índice 4 en nuestro esquema) corresponde a un ID diferente en los 80 de COCO, lo que explica el resultado nulo. El fine-tuned, entrenado con el mapeo correcto, sí aprende la clase bus correctamente.

La clase 'truck' pasa de no detectarse (AP=0 en genérico por el bug) a AP50=0.3179 en el fine-tuned. Aún es bajo — los camiones desde atrás siguen siendo difíciles — pero el modelo ahora al menos los detecta.

Limitación principal del entrenamiento: los datos de COCO son escenas internacionales, no españolas. Un fine-tuning adicional con imágenes de dashcam en autopistas españolas mejoraría significativamente la detección en el contexto real de uso de la aplicación.

9. Conclusiones y trabajo futuro

Logros del proyecto	Limitaciones conocidas
Pipeline completo de visión artificial: datos → preprocesado → entrenamiento → inferencia → aplicación web	Señales de velocidad en condiciones nocturnas extremas o muy borrosas pueden no detectarse (OCR falla).
Detección de 6 clases con corrección de IDs COCO (bug crítico resuelto)	La estimación de velocidad requiere calibración para cada cámara concreta (la perspectiva varía por ángulo/altura).
Estimación de velocidad con corrección de perspectiva por vehículo	Datos de entrenamiento en COCO (escenas internacionales, no españolas). Vistas traseras de camiones son escasas.
Detección de señales de velocidad mediante HSV + contornos + OCR	El modelo YOLO no detecta motos de reparto ni furgonetas con alta fiabilidad desde ángulos poco comunes.
Lectura de carteles de dirección con EasyOCR y validación por texto	La detección de señales es puramente clásica (HSV+OCR), no neuronal. Sensible a iluminación atípica.
Modelo fine-tuned propio (best.pt) que mejora bus +753% AP50	
Interfaz web completa con heatmap, línea de conteo, informes PDF/CSV	

Trabajo futuro	Resumen ejecutivo
1. Anotar imágenes propias de carretera española con Roboflow y reentrenar.	Smart Traffic Analyzer demuestra que es posible construir un sistema completo de análisis de tráfico con herramientas open-source, un Mac M4 Pro y aproximadamente una semana de desarrollo.
2. Sustituir detección clásica de señales por un modelo YOLO especializado en señales de tráfico europeas.	~2.500 Líneas de código 6
3. Calibración automática de velocidad: usar marcas del carril (3m estándar) como referencia de escala.	Clases detectadas 76 min Tiempo entreno
4. Añadir detección de comportamientos: cambio de carril, distancia de seguridad.	Archivos Python 8+
5. Exportar modelo a ONNX/TensorRT para inferencia en tiempo real en GPU.	Bugs resueltos 48.1%
6. Integración con cámara IP fija para monitorización permanente de tramo.	mAP50 alcanzado